DOCKER NETWORKING WITH ISOLATED CONTAINERS - SECUREBRIDGE

A MICRO PROJECT

REPORT

(22ITF02 – DEVOPS)

Submitted by

GOWRI D

(23ITR048)

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

(Autonomous)

PERUNDURAI, ERODE – 638 060

APRIL 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI, ERODE – 638060

APRIL 2026

BONAFIDE CERTIFICATE

This is to certify that this micro project report entitled **"DOCKER NETWORKING WITH ISOLATED CONTAINERS - SECUREBRIDGE"** is the bonafide work carried out by **GOWRI D (23ITR048)** in partial fulfilment of the requirements for the course **22ITF02 – DEVOPS** during the academic year **2025 – 2026**, and that the work has been carried out under my supervision and has not been submitted elsewhere for any other course or award.

FACULTY IN-CHARGE **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the Final Project Review for the course 22ITF02 – DEVOPS held on

_____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENTS

CHAPTER No	TITLE	PAGE NO
1	**AIM**	1
2	**ABSTRACT**	1
3	**SCOPE AND OBJECTIVES**	2
4	**SYSTEM SPECIFICATIONS**	3
	4.1 HARDWARE REQUIREMENTS	3
	4.2 SOFTWARE REQUIREMENTS	3
5	**PROJECT DESIGN**	4
6	**ADVANTAGES**	5
7	**PROPOSED SYSTEM**	5
8	**IMPLEMENTATION OF THE PROJECT**	6
9	**OUTPUT**	7
10	**CONCLUSION**	9
11	**COMMANDS / CODING**	9
12	**CERTIFICATE**	12
13	**REFERENCES**	12

DOCKER NETWORKING WITH ISOLATED CONTAINERS - SECUREBRIDGE

AIM

The aim of this project is to implement secure container communication using Docker networking with isolated containers. It includes creating custom Docker networks for the frontend, backend, and database to control communication between services. Proper network isolation ensures that only authorized containers can communicate with each other. This setup improves security, simplifies deployment, and ensures efficient management of the application.

ABSTRACT

This project focuses on designing and implementing a secure container networking system using Docker with isolated containers, where the entire system is divided into multiple independent yet controlled services such as a frontend application for user interaction, a backend API to handle business logic, and a database to store application data. Custom Docker networks are used to isolate containers while allowing only authorized communication between selected services. The docker-compose.yml file is used to define, configure, and run all services together as a single cohesive application, reducing the complexity of manual setup and improving development workflow. It provides built-in networking that ensures secure and controlled communication between containers, enabling the frontend to interact with the backend and the backend to access the database safely. Overall, this project demonstrates how Docker networking with isolated containers enhances security, simplifies deployment, and provides an efficient structure for managing modern containerized applications.

SCOPE AND OBJECTIVES

SCOPE:

- Focuses on implementing Docker networking with isolated containers for secure communication between services.
- Covers the complete setup of frontend, backend API, and database as separate containers with controlled access.
- Demonstrates how to create Docker images for each service and define them in a docker-compose.yml file.
- Explains container networking using custom networks to enable secure and selective communication.
- Emphasizes security, scalability, and efficient service management using containerization.

OBJECTIVES :

- To create Docker images for frontend, backend API, and database services to ensure consistency and portability.
- To write and configure a docker-compose.yml file to manage multiple containers as a single unified system.
- To create custom Docker networks to isolate containers and control communication between services.
- To enable communication only between selected containers such as backend and database.
- To test and verify network isolation to ensure secure and controlled interaction between services.

SYSTEM SPECIFICATIONS

Hardware Requirements:

- A stable internet connection to download and configure tools such as Docker.
- Minimum 4 GB RAM (8 GB recommended for better performance) to run Docker containers smoothly without performance issues.
- At least 2 GB of free storage space to install required tools, store project files, and manage Docker images and containers.
- A system with a modern processor (Intel i3 or above) and a screen size of 13" or larger with at least 1366×768 resolution for comfortable development and monitoring.

Software Requirements:

- **Windows** – To develop and run the project environment and manage all tools locally.
- **Docker** – To create and run containers and manage networking between services.
- **Command Prompt / Terminal** – To execute Docker commands for setup and deployment.
- **Google Chrome (or any browser**) – To access and test the application.
- **Visual Studio Code** – To write, edit, and manage Dockerfiles, docker-compose.yml, and application code efficiently.

PROJECT DESIGN

• The project sets up a multi-container application environment for Docker Networking with Isolated Containers using Docker and Docker Compose on a local system.

• The application is divided into three main services: a frontend, a backend API, and a database, each running in separate containers.

• Docker is used to create images for each service, ensuring consistency and portability across different environments.

• Docker Compose is used to define and manage all services in a single docker-compose.yml file, simplifying the deployment process.

• Container networking is configured using custom networks to isolate services while allowing controlled communication between selected containers.

• The backend API handles business logic and acts as a bridge between the frontend and the isolated database, while the database stores application data securely.

• Volume mapping is used to persist database data even when containers are restarted or removed.

• The setup is secure, efficient, and easy to manage, making it suitable for development and testing environments.

• The final outcome is a fully functional multi-container application that demonstrates Docker networking, service isolation, and secure communication between containers.

ADVANTAGES

- Simplifies management of multiple containers using Docker Compose.
- Reduces complexity by organizing services with isolated networking.
- Ensures consistency across environments through containerization.
- Enhances security by isolating database and restricting direct access.
- Enables controlled communication between selected containers.
- Improves development efficiency and faster deployment process.
- Provides isolated environments for each service, reducing conflicts.
- Helps in understanding real-world Docker networking and DevOps practices.

PROPOSED SYSTEM

- Uses Docker Desktop to create and manage containers for frontend, backend API, and database in a simple and user-friendly environment.
- Implements Docker Compose to define and run multiple services using a docker-compose.yml file.
- Structures the application into separate containers (frontend, backend, database) to maintain proper isolation and modularity.
- Utilizes custom Docker networks to control communication between containers and ensure secure access.
- Supports volume mapping to store and manage persistent database data efficiently.
- Provides a secure, efficient, and easy-to-manage setup suitable for development and testing on a local system.

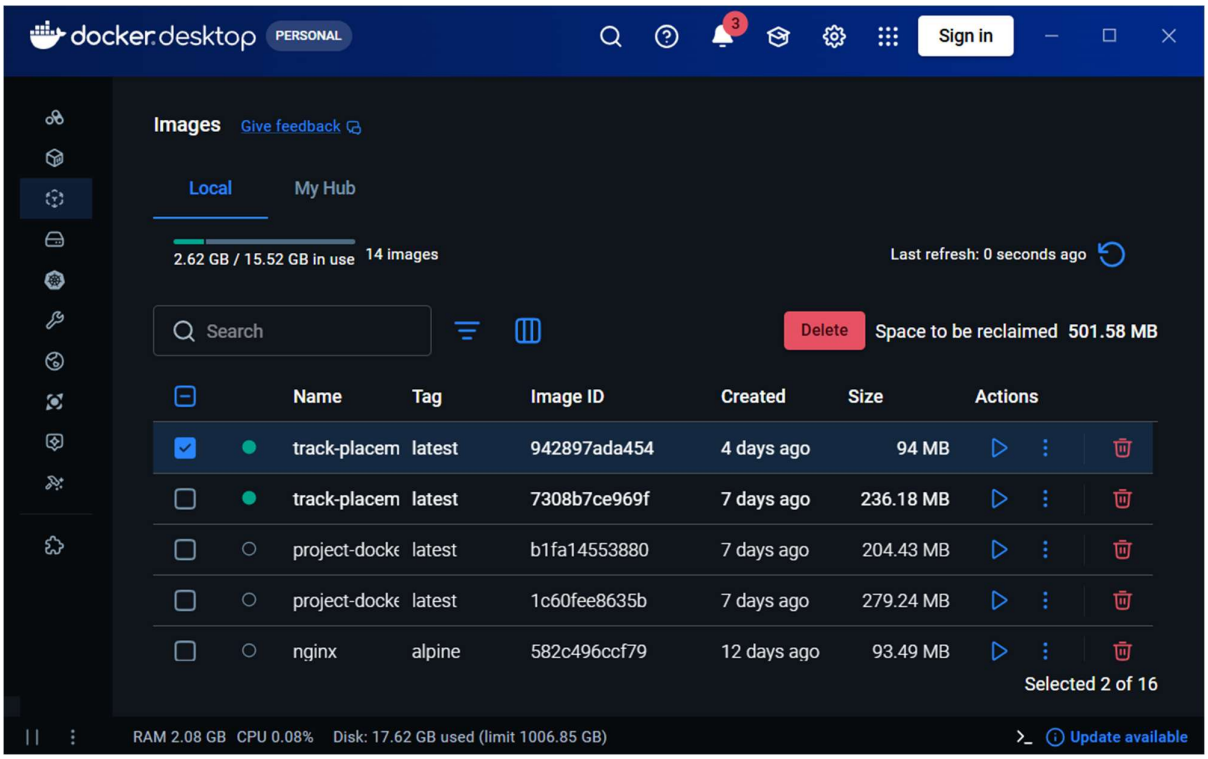
IMPLEMENTATION OF THE PROJECT

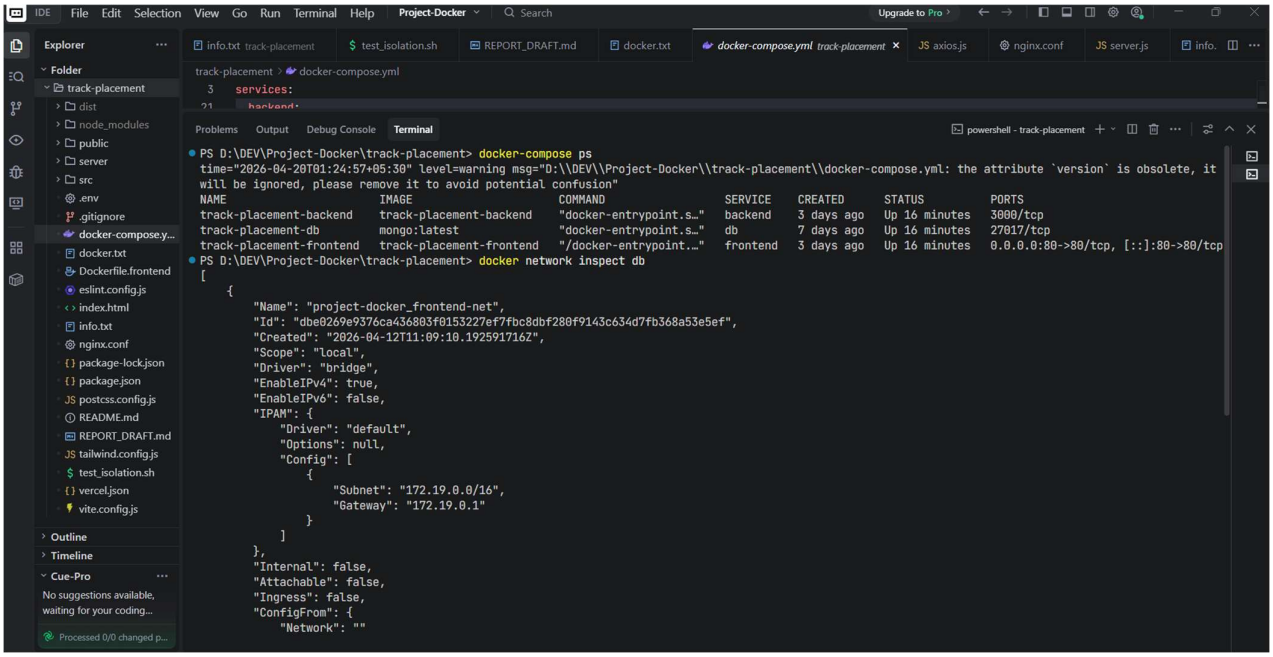
The project was implemented by setting up a local development environment using Docker Desktop on a Windows system. A project folder was created to organize all application files, including frontend, backend API, and database configurations. Dockerfiles were written for each service to build custom images with required dependencies.

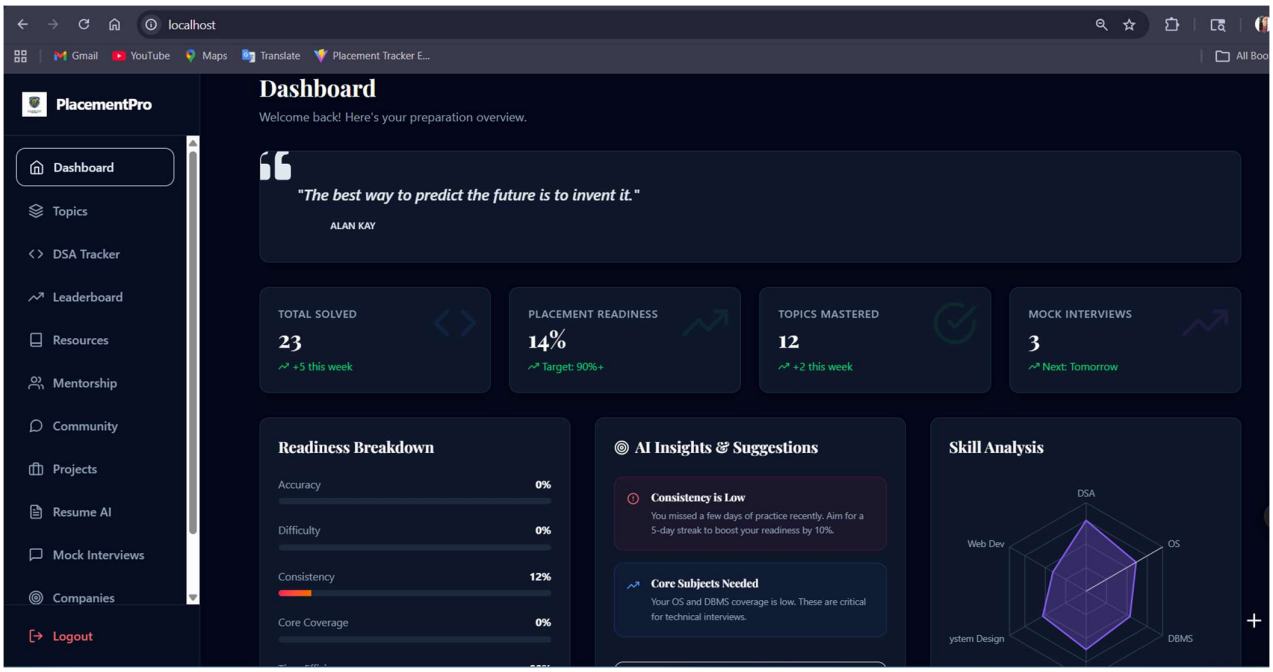
A docker-compose.yml file was then created to define all services, including their ports, volumes, and custom network configurations for isolation. Using Docker Compose through Docker Desktop, all containers were built and started together with a single command. The frontend container communicates with the backend API, which further interacts with the database container through controlled Docker networks.

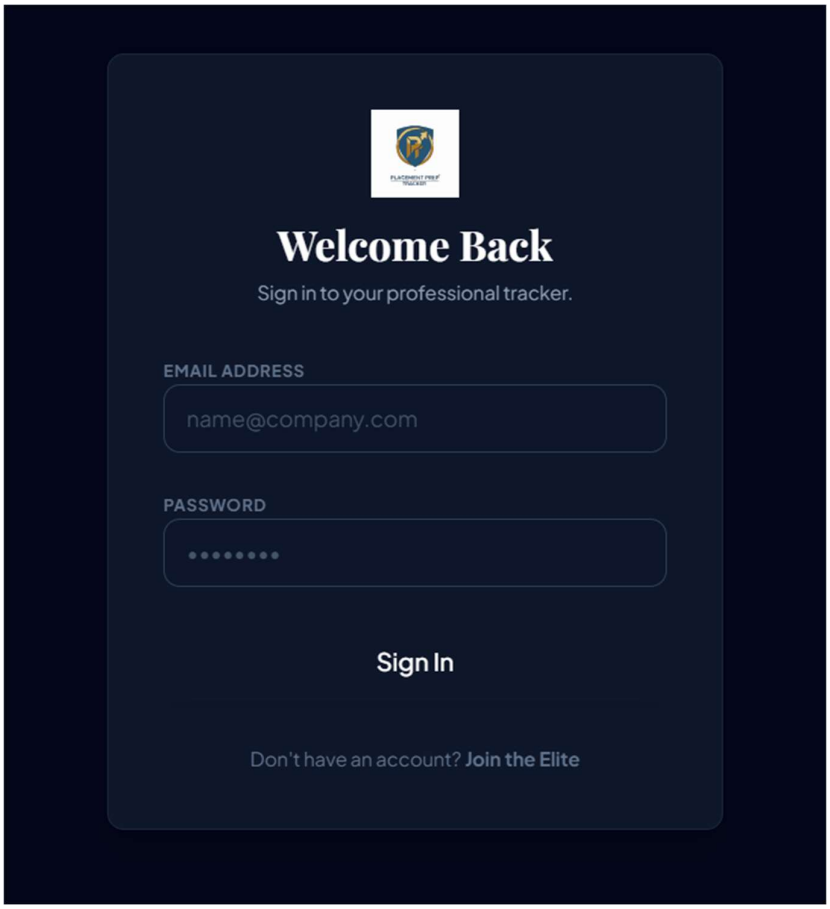
Finally, the application was tested by running network checks and verifying that only allowed containers could communicate, ensuring proper isolation. Changes were easily applied by rebuilding containers through Docker Desktop. The result is a secure multi-container application that is easy to deploy, manage, and maintain.

OUTPUT


docker desktop
PERSONAL
Sign in
Images
Give feedback
Local
My Hub
2.62 GB / 15.52 GB in use
14 images
Last refresh: 0 seconds ago
Search
Delete
Space to be reclaimed 501.58 MB
Name Tag Image ID Created Size Actions
track-placem latest 942897ada454 4 days ago 94 MB
track-placem latest 7308b7ce969f 7 days ago 236.18 MB
project-docke latest b1fa14553880 7 days ago 204.43 MB
project-docke latest 1c60fee8635b 7 days ago 279.24 MB
nginx alpine 582c496ccf79 12 days ago 93.49 MB
Selected 2 of 16
RAM 2.08 GB CPU 0.08% Disk: 17.62 GB used (limit 1006.85 GB)
Update available


PS D:\DEV\Project-Docker\track-placement> docker-compose ps
time="2026-04-20T01:24:57+05:30" level=warning msg="D:\\DEV\\Project-Docker\\track-placement\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
track-placement-backend track-placement-backend "docker-entrypoint.s…" backend 3 days ago Up 16 minutes 3000/tcp
track-placement-db mongo:latest "docker-entrypoint.s…" db 7 days ago Up 16 minutes 27017/tcp
track-placement-frontend track-placement-frontend "/docker-entrypoint.…" frontend 3 days ago Up 16 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp
PS D:\DEV\Project-Docker\track-placement> docker network inspect db
[
{
"Name": "project-docker_frontend-net",
"Id": "dbe0269e9376ca436803f0153227ef7fbc8dbf280f9143c634d7fb368a53e5ef",
"Created": "2026-04-12T11:09:10.192591716Z",
"Scope": "local",
"Driver": "bridge",
"EnableIPv4": true,
"EnableIPv6": false,
"IPAM": {
"Driver": "default",
"Options": null,
"Config": [
{
"Subnet": "172.19.0.0/16",
"Gateway": "172.19.0.1"
}
]
},
"Internal": false,
"Attachable": false,
"Ingress": false,
"ConfigFrom": {
"Network": ""


localhost
Gmail
YouTube
Maps
Translate
Placement Tracker E...
PlacementPro
Dashboard
Topics
DSA Tracker
Leaderboard
Resources
Mentorship
Community
Projects
Resume AI
Mock Interviews
Companies
Logout
Dashboard
Welcome back! Here's your preparation overview.
"The best way to predict the future is to invent it."
ALAN KAY
TOTAL SOLVED
23
+5 this week
PLACEMENT READINESS
14%
Target: 90%+
TOPICS MASTERED
12
+2 this week
MOCK INTERVIEWS
3
Next: Tomorrow
Readiness Breakdown
Accuracy 0%
Difficulty 0%
Consistency 12%
Core Coverage 0%
AI Insights & Suggestions
Consistency is Low
You missed a few days of practice recently. Aim for a 5-day streak to boost your readiness by 10%.
Core Subjects Needed
Your OS and DBMS coverage is low. These are critical for technical interviews.
Skill Analysis
DSA
OS
DBMS
Web Dev


Welcome Back
Sign in to your professional tracker.
EMAIL ADDRESS
name@company.com
PASSWORD
Sign In
Don't have an account? Join the Elite

CONCLUSION

Using Docker Compose along with Docker Desktop provides an effective solution for implementing Docker networking with isolated containers. Docker is used to containerize the frontend, backend API, and database, ensuring consistency and portability across different environments. Docker Compose simplifies the process by allowing all services to be defined and executed together through a single configuration file. Custom networking enables controlled communication between containers while keeping critical services like the database isolated and secure. This setup is simple, efficient, and easy to manage, making it ideal for development and testing. Overall, the project demonstrates modern DevOps practices, enabling secure communication, reduced complexity, improved scalability, and better system reliability.

COMMANDS

1. **Navigate to Project Directory**
   cd C:\devops
   Opens your project folder containing Dockerfile and docker-compose.yml

2. **Build Docker Images**
   docker-compose build
   Builds images for frontend, backend API, and database services

3. **Run Multi-Container Application**
   docker-compose up -d
   Starts all containers together with defined networks

4. **Check Running Containers**
   docker ps
   Displays all running containers (frontend, backend, database)

5. **Check Networks**
   docker network ls
   Shows custom networks created for isolation

6. **Inspect Network Isolation**
   docker network inspect <network_name>
   Verifies which containers are connected to each network

7. **Stop Application**
   docker-compose down
   Stops and removes all running containers

8. **Access Application**
   http://localhost:3000
   (configured port) — Opens the application in browser

CODING

Docker-compose.yml

```
version: '3.8'

services:
  # 1. FRONTEND TIER (Nginx + React App)
  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    container_name: track-placement-frontend
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - app-network
    depends_on:
      - backend

  # 2. BACKEND TIER (Node.js API)
  backend:
    build:
      context: ./server
      dockerfile: Dockerfile
    container_name: track-placement-backend
    restart: unless-stopped
    environment:
      - NODE_ENV=production
      - MONGO_URI=mongodb://db:27017/track-placement
      - PORT=3000
    networks:
      - app-network
      - db-network
    depends_on:
      - db
```

```
  # 3. DATABASE TIER (MongoDB)
  db:
    image: mongo:latest
    container_name: track-placement-db
    restart: unless-stopped
    volumes:
      - db-data:/data/db
    networks:
      - db-network
    # DATABASE IS ISOLATED - NO PORTS EXPOSED TO HOST

networks:
  # Multi-tier Network Isolation
  app-network:
    driver: bridge
  db-network:
    driver: bridge

volumes:
  # Secure Data Persistence
  db-data:
    driver: local
```

CERTIFICATE

GUVI | HCL
Skill Up. Level Up

Google for Education Partner

ISO 9001-27001 CERTIFIED

Certificate of Completion

Gowri D

is here by awarded the certificate of achievement
for the successful completion of

Mastering DevOps

M. Arunprakash
Founder and CEO,
GUVI Geek Networks.

Certificate issued on : March 30 2026

Valid certificate ID : 7zP9178ov04k8m6B41

Verify certificate at: www.guvi.in/certificate?id=7zP9178ov04k8m6B41

REFERENCES

• Docker Inc. (Official Docker Documentation) – https://docs.docker.com/

• Docker Compose Documentation – https://docs.docker.com/compose/

• GUVI (DevOps & Docker Tutorials) – https://www.guvi.in

• GeeksforGeeks (Docker & DevOps Articles) – https://www.geeksforgeeks.org/devops/

• W3Schools (Web Development Basics) – https://www.w3schools.com

• Microsoft Learn (Docker & Container Guides) – https://learn.microsoft.com/en-us/virtualization/docker/

• GitHub (Sample Docker Projects & Code) – https://github.com/

• TutorialsPoint (Docker Tutorial) – https://www.tutorialspoint.com/docker/index.htm

• FreeCodeCamp (Docker & DevOps Learning) – https://www.freecodecamp.org/news

• Kubernetes Documentation (Container Orchestration Basics) – https://kubernetes.io/docs/

• Nginx Official Documentation – https://nginx.org/en/docs/